

Arquitectura, Compilación y Seguridad en C++

Documentación técnica de la cadena de herramientas (Toolchain), modelo de enlaces, depuración y mejores prácticas de seguridad en tiempo de ejecución

MANUAL COMPLETO DE REFERENCIA • PRINCIPIANTES & DESARROLLADORES AVANZADOS

MÓDULO 1: Cadena de Herramientas, Proceso de Compilación y Estructura de Proyectos

01. Preprocessor (Preprocesador)

BUILD & TOOLCHAIN

Primera fase del pipeline de compilación. El preprocesador realiza manipulación puramente textual del código antes de la compilación real. Modifica el código reemplazando macros (`#define`), expandiendo directivas de inclusión (`#include`), evaluando compilación condicional (`#ifdef` , `#ifndef`) y quitando comentarios.

```
#define TAMANO_MAXIMO 100
#include <iostream>
#ifdef DEBUG_MODE
#define LOG(x) std::cout << x << std::endl
#else
#define LOG(x)
#endif

int main() {
    LOG("Modo Depuración");
    return 0;
}
```

02. Translation Units (Unidades de Traducción)

BUILD & TOOLCHAIN

Una Unidad de Traducción es el resultado de procesar un archivo fuente (.cpp) junto con todos los encabezados (.hpp) e inclusiones directas o indirectas expandidos por el preprocesador. Es el bloque individual de código que entra al compilador.

```
// Archivo: main.cpp
#include "math_utils.hpp" // El preprocesador expande math_utils.hpp aquí

// La combinación de main.cpp + el contenido insertado de math_utils.hpp
// constituye una única Unidad de Traducción (Translation Unit).
```

03. Compiler (Compilador)

BUILD & TOOLCHAIN

Etapas que traduce la Unidad de Traducción (C++ alto nivel) a código de lenguaje ensamblador específico para la arquitectura de destino (x86, ARM, RISC-V). Analiza sintaxis, realiza chequeos de tipo, construye el AST (Abstract Syntax Tree) y aplica optimizaciones.

```
// Comando típico de invocación del compilador C++ (GCC / Clang)
// g++ -S main.cpp -o main.s
// Genera 'main.s' conteniendo código en lenguaje ensamblador.
```

04. Assembler (Ensamblador)

BUILD & TOOLCHAIN

Herramienta de la cadena que convierte las instrucciones en lenguaje ensamblador (texto legible) generadas por el compilador en instrucciones de código máquina binario directo, empaquetándolas en un archivo objeto.

```
// Comando típico de ensamblado
// as main.s -o main.o
// O directamente con el driver del compilador:
// g++ -c main.cpp -o main.o
```

05. Object Files (Archivos Objeto)

BUILD & TOOLCHAIN

Archivos binarios intermedios (extensión `.o` en Linux/macOS, `.obj` en Windows) producidos por el ensamblador. Contienen código máquina, secciones de datos y una **tabla de símbolos** (funciones y variables declaradas o requeridas), pero aún no son ejecutables.

```
// Inspección de la tabla de símbolos de un archivo objeto en Linux/macOS
// nm main.o
// Muestra símbolos definidos (T) y no resueltos (U - Unresolved)
```

06. Linker (Enlazador)

BUILD & TOOLCHAIN

Herramienta responsable de combinar uno o múltiples archivos objeto y librerías en un único archivo ejecutable o librería final. Su tarea central es la **resolución de símbolos**: conectar cada llamada a función con su dirección de memoria real.

```
// Invocación del enlazador mediante g++
// g++ main.o math_utils.o -o mi_programa
// Combina los archivos objeto en el ejecutable 'mi_programa'.
```

07. Static Libraries (Librerías Estáticas)

BUILD & TOOLCHAIN

Colección empaquetada de archivos objeto (extensión `.a` en Linux/macOS, `.lib` en Windows). Durante el enlazado, el código binario necesario de la librería se copia e integra directamente dentro del ejecutable final.

```
// Creación de una librería estática con 'ar' (archiver)
// ar rcs libmi_lib.a modulo1.o modulo2.o

// Enlazado de la librería estática
// g++ main.o -L. -lmi_lib -o ejecutable
```

08. Dynamic Libraries (Librerías Dinámicas)

BUILD & TOOLCHAIN

Librerías vinculadas en tiempo de ejecución o carga (`.so` en Linux, `.dll` en Windows, `.dylib` en macOS). El ejecutable contiene solo referencias a la librería; el código real se comparte e incluye en memoria en tiempo de ejecución.

```
// Creación de librería dinámica en Linux
// g++ -shared -fPIC modulo1.o -o libmi_lib.so

// El ejecutable requiere encontrar 'libmi_lib.so' al ejecutarse (LD_LIBRARY_PATH)
```

09. Executables (Ejecutables)

BUILD & TOOLCHAIN

Archivo binario autónomo estructurado en formatos estándar (ELF en Linux, PE en Windows, Mach-O en macOS) que el sistema operativo puede cargar directamente en memoria RAM para su ejecución por la CPU.

```
// Ejecución en terminal
// ./mi_programa

// Formato binario ELF/PE que incluye punto de entrada (main),
// sección .text (código), .data (variables) y .bss
```

10. Debug Symbols (Símbolos de Depuración)

BUILD & TOOLCHAIN

Información metadatos agregada al binario (formatos DWARF o PDB) que asigna direcciones de memoria de código máquina con líneas de código fuente C++, nombres de variables y firmas de funciones. Esenciales para depurar con [gdb](#) o [lldb](#).

```
// Compilación agregando símbolos de depuración (-g)
// g++ -g main.cpp -o programa_debug

// Permite pausar el programa en gdb y ver variables por nombre original
// (gdb) break main.cpp:15
// (gdb) print mi_variable
```

11. Debug Build (Compilación Debug)

BUILD & TOOLCHAIN

Configuración de compilación orientada al desarrollo. Desactiva optimizaciones del compilador ([-O0](#)) para mantener una correspondencia directa línea a línea con el código fuente e incluye símbolos de depuración y macros [assert](#) activas.

```
// Comando para build de depuración
// g++ -O0 -g -DDEBUG main.cpp -o App_Debug

// Genera un ejecutable más lento e ineficiente pero 100% depurable.
```

12. Release Build (Compilación Release)

BUILD & TOOLCHAIN

Configuración de compilación orientada a producción. Habilita optimizaciones agresivas del compilador ([-O2](#) o [-O3](#)) como inlining de funciones, desenrollado de bucles y vectorización SIMD, removiendo aserciones e información no esencial.

```
// Comando para build de producción/release
// g++ -O3 -DNDEBUG main.cpp -o App_Release

// Genera un ejecutable altamente optimizado en tamaño y velocidad.
```

13. Include Paths (Rutas de Inclusión)

BUILD & TOOLCHAIN

Directorios pasados al compilador mediante el flag [-I](#) en los que el preprocesador buscará los archivos de cabecera ([#include <header.hpp>](#)) si no se encuentran en los directorios estándar del sistema.

```
// Especificar rutas de búsqueda para headers
// g++ -I./incluir -I/usr/local/include/subsistema main.cpp -o app
```

14. Library Paths (Rutas de Librerías)

BUILD & TOOLCHAIN

Opciones de compilación para indicar dónde encontrar librerías compiladas. El flag `-L` añade directorios de búsqueda de librerías, mientras que `-l` especifica el nombre de la librería a enlazar.

```
// -L especifica la ruta de carpetas de librerías
// -lmath especifica enlazar 'libmath.a' o 'libmath.so'
// g++ main.o -L./librerías -lmath -o app
```

15. Linking Process (Proceso de Enlazado)

BUILD & TOOLCHAIN

Fase final donde el linker une los binarios intermedios. Si un archivo llama a una función declarada pero no definida en su TU, el enlazador busca esa definición en otros archivos objeto o librerías. Si no la encuentra, genera un error *'Undefined Reference'*.

```
// Ejemplo de error común de enlazado (Undefined Reference):
// main.cpp llama a 'void procesar()', pero procesar() nunca fue definida.
// Error en linker: undefined reference to `procesar()'
```

MÓDULO 2: Manejo de Errores, Robustez y Seguridad en C++

16. Exceptions (Excepciones)

SEGURIDAD & RUNTIME

Mecanismo de C++ para notificar y manejar situaciones anómalas en tiempo de ejecución. Permite alterar el flujo del programa transfiriendo el control desde el punto de error hasta el manejador de captura más cercano.

```
#include <iostream>
#include <stdexcept>

double dividir(double a, double b) {
    if (b == 0.0) throw std::runtime_error("División por cero");
    return a / b;
}
```

17. try / catch

SEGURIDAD & RUNTIME

Estructura de bloques para capturar excepciones. El código propenso a fallos se encierra en un bloque `try`. Si se lanza (`throw`) una excepción, la ejecución salta de inmediato al bloque `catch` correspondiente.

```
#include <iostream>
#include <stdexcept>

int main() {
    try {
        double res = dividir(10.0, 0.0);
    } catch (const std::exception& e) {
        std::cout << "Error capturado: " << e.what() << "\n";
    }
    return 0;
}
```

18. `std::exception`

SEGURIDAD & RUNTIME

Clase base estándar de la jerarquía de excepciones en C++ (`<stdexcept>`). Ofrece el método virtual `what()` para obtener la descripción textual del error. Subclases comunes: `std::runtime_error`, `std::invalid_argument`, `std::out_of_range`.

```
#include <iostream>
#include <stdexcept>

class MiExcepcion : public std::exception {
public:
    const char* what() const noexcept override {
        return "Error personalizado en MiExcepcion";
    }
};
```

19. `assert`

SEGURIDAD & RUNTIME

Macro de verificación en tiempo de ejecución (`<cassert>`). Evalúa una expresión lógica; si es falsa, detiene de inmediato la ejecución del programa e imprime detalles del fallo. Desactivada automáticamente en builds Release (`-DNDEBUG`).

```
#include <iostream>
#include <cassert>

void procesar_edad(int edad) {
    assert(edad >= 0 && "La edad no puede ser negativa");
    std::cout << "Edad procesada: " << edad << "\n";
}

int main() {
    procesar_edad(20);
    // procesar_edad(-5); // Falla el assert y aborta la ejecución
    return 0;
}
```

20. `static_assert`

SEGURIDAD & RUNTIME

Mecanismo de aserción en tiempo de compilación (C++11). Evalúa una condición constante durante la compilación. Si la condición es falsa, la compilación se detiene de inmediato generando un error con el mensaje indicado.

```
#include <iostream>
#include <type_traits>

template <typename T>
void verificar_tipo() {
    static_assert(std::is_integral<T>::value, "T debe ser un tipo entero");
}

int main() {
    verificar_tipo<int>(); // Compila bien
    // verificar_tipo<double>(); // Error de compilación
    return 0;
}
```

21. Error Codes (Códigos de Error)

SEGURIDAD & RUNTIME

Estrategia explícita de manejo de errores donde las funciones retornan un valor que indica éxito o el tipo de fallo. Utilizada ampliamente en sistemas de alto rendimiento o donde las excepciones están desactivadas.

```
#include <iostream>

enum class ErrorCode { OK, FileNotFound, AccessDenied };

ErrorCode abrir_archivo(const std::string& ruta) {
    if (ruta.empty()) return ErrorCode::FileNotFound;
    return ErrorCode::OK;
}

int main() {
    if (abrir_archivo("") != ErrorCode::OK) {
        std::cout << "Fallo al abrir archivo.\n";
    }
    return 0;
}
```

22. Undefined Behavior (Comportamiento Indefinido - UB)

SEGURIDAD & RUNTIME

Situación en C++ donde el estándar no impone ningún comportamiento sobre la ejecución del programa tras violar sus reglas. Un programa con UB puede funcionar por azar, colapsar, corromper memoria o producir resultados inconsistentes según el compilador y nivel de optimización.

```
#include <iostream>

int main() {
    int x; // Variable no inicializada (contiene basura de memoria)
    // Usar 'x' sin inicializar provoca Undefined Behavior (UB)
    std::cout << x << "\n";

    int arr[2] = {1, 2};
    // Acceso fuera de rango provoca UB
    int val = arr[5];
    return 0;
}
```

23. Null Pointers (Punteros Nulos)

SEGURIDAD & RUNTIME

Puntero que no apunta a ninguna dirección de memoria válida (`nullptr` en C++11). Intentar desreferenciar (`*ptr`) o acceder a miembros de un puntero nulo genera un fallo de segmentación (Segmentation Fault / Access Violation).

```
#include <iostream>

int main() {
    int* ptr = nullptr; // Puntero nulo seguro

    if (ptr != nullptr) {
        std::cout << *ptr << "\n";
    } else {
        std::cout << "Puntero nulo detectado con seguridad.\n";
    }

    // *ptr = 10; // Causa Crash (Segmentation Fault)
    return 0;
}
```

24. Dangling Pointers (Punteros Colgantes)

SEGURIDAD & RUNTIME

Puntero que continúa almacenando la dirección de memoria de un objeto cuya memoria ya ha sido liberada o cuyo tiempo de vida (lifetime) ha finalizado. Desreferenciarlo causa Comportamiento Indefinido.

```
#include <iostream>

int* crear_puntero_colgante() {
    int x = 42;
    return &x; // Error: 'x' se destruye al salir de la función
}

int main() {
    int* ptr = crear_puntero_colgante();
    // 'ptr' es ahora un puntero colgante (dangling pointer)
    // std::cout << *ptr; // UB: acceso a memoria inválida
    return 0;
}
```

25. Buffer Overflow (Desbordamiento de Búfer)

SEGURIDAD & RUNTIME

Vulnerabilidad grave de seguridad producida al escribir más datos de los que un bloque de memoria contigua (búfer) puede albergar. Sobrescribe posiciones adyacentes de la memoria, corrompiendo variables o la pila de llamadas.

```
#include <iostream>
#include <cstring>

void inseguro() {
    char buffer[8];
    // Escribir más de 8 bytes sobrepasa el buffer
    // std::strcpy(buffer, "TextoDemasiadoLargoParaElBuffer"); // Buffer Overflow
}

int main() {
    inseguro();
    return 0;
}
```

26. Out-of-Bounds Access (Acceso Fuera de Límites)

SEGURIDAD & RUNTIME

Lectura o escritura en un índice fuera del rango válido de un arreglo o contenedor. En arreglos C-style o `std::vector` con operador `[]`, causa Comportamiento Indefinido. Para un acceso seguro con verificación de límites se debe usar `.at()`.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> nums = {10, 20, 30};

    try {
        // .at(index) lanza std::out_of_range si el índice es inválido
        std::cout << nums.at(5) << "\n";
    } catch (const std::out_of_range& e) {
        std::cout << "Capturado acceso fuera de rango: " << e.what() << "\n";
    }

    return 0;
}
```